

CO4-AT1-CODING CHALLENGE

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

Aim

To simulate the IPSec AH process by generating a MAC for an IP packet and verifying its integrity.

CODING:

```
#include <iostream>

#include <string>

#include <functional>

using namespace std;

string generateMAC(string packet, string key)
{
    hash<string> h;

    size_t value = h(packet + key);

    return to_string(value);
}

int main()
{
    string packet, key, receivedMAC;

    cout << "Enter IP Packet: ";

    getline(cin, packet);

    cout << "Enter Secret Key: ";
```

```

getline(cin, key);

string mac = generateMAC(packet, key);

cout << "\n--- IPSec AH Simulation ---\n";

cout << "Original Packet : " << packet << endl;

cout << "Generated MAC : " << mac << endl;

cout << "\nEnter MAC for verification: ";

getline(cin, receivedMAC);

if (receivedMAC == mac)

    cout << "Verification Successful - Packet Integrity Valid\n";

else

    cout << "Verification Failed - Packet Modified\n";

return 0;

}

```

OUTPUT:

The screenshot shows a C++ IDE with a source code editor on the left and an output window on the right. The source code is a C++ program that simulates IPSec AH. It prompts the user to enter an IP packet and a secret key, then generates a MAC for the packet. It then prompts the user to enter a MAC for verification. If the entered MAC matches the generated MAC, it prints "Verification Successful - Packet Integrity Valid". Otherwise, it prints "Verification Failed - Packet Modified".

```

main.cpp
1  #include <iostream>
2  #include <string>
3  #include <functional>
4  using namespace std;
5  string generateMAC(string packet, string key)
6  {
7      hash<string> h;
8      size_t value = h(packet + key);
9      return to_string(value);
10 }
11 int main()
12 {
13     string packet, key, receivedMAC;
14     cout << "Enter IP Packet: ";
15     getline(cin, packet);
16     cout << "Enter Secret Key: ";
17     getline(cin, key);
18     string mac = generateMAC(packet, key);
19     cout << "\n--- IPSec AH Simulation ---\n";
20     cout << "Original Packet : " << packet << endl;
21     cout << "Generated MAC : " << mac << endl;
22     cout << "\nEnter MAC for verification: ";
23     getline(cin, receivedMAC);
24     if (receivedMAC == mac)
25         cout << "Verification Successful - Packet Integrity Valid\n";
26     else
27         cout << "Verification Failed - Packet Modified\n";
28     return 0;
}

```

Output

```

Enter IP Packet: Hello IP Packet
Enter Secret Key: secret123

--- IPSec AH Simulation ---
Original Packet : Hello IP Packet
Generated MAC : 8830991239118338748

Enter MAC for verification: 8830991239118338748
Verification Successful - Packet Integrity Valid

--- Code Execution Successful ---

```

```

main.cpp
1  #include <string>
2  #include <functional>
3  using namespace std;
4  string generateMAC(string packet, string key)
5  {
6      hash<string> h;
7      size_t value = h(packet + key);
8      return to_string(value);
9  }
10 int main()
11 {
12     string packet, key, receivedMAC;
13     cout << "Enter IP Packet: ";
14     getline(cin, packet);
15     cout << "Enter Secret Key: ";
16     getline(cin, key);
17     string mac = generateMAC(packet, key);
18     cout << "\n--- IPsec AH Simulation ---\n";
19     cout << "Original Packet : " << packet << endl;
20     cout << "Generated MAC   : " << mac << endl;
21     cout << "\nEnter MAC for verification: ";
22     getline(cin, receivedMAC);
23     if (receivedMAC == mac)
24         cout << "Verification Successful - Packet Integrity Valid\n";
25     else
26         cout << "Verification Failed - Packet Modified\n";
27     return 0;
28 }

```

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

AIM:

To simulate ESP encryption and decryption of an IP packet payload.

CODING:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
string encryptData(string data, char key)
```

```
{
```

```
    for (char &c : data)
```

```
        c = c ^ key; return data;
```

```
}
```

```

int main()
{
    string payload;

    char ke

    cout << "Enter IP Packet Payload: ";

    getline(cin, payload);

    cout << "Enter Encryption Key (single character): ";

    cin >> key;

    string encrypted = encryptData(payload, key);

    cout << "\n--- IPSec ESP Simulation ---\n";

    cout << "Original Payload : " << payload << endl;

    cout << "Encrypted Payload: " << encrypted << endl;

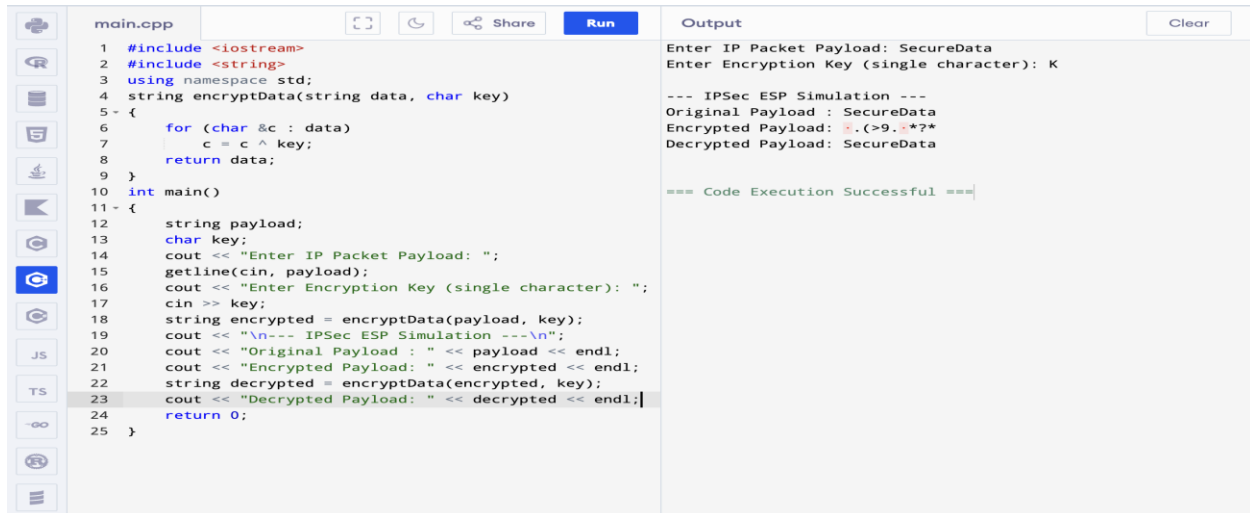
    string decrypted = encryptData(encrypted, key);

    cout << "Decrypted Payload: " << decrypted << endl;

    return 0;
}

```

OUTPUT:



```
main.cpp
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 string encryptData(string data, char key)
5 {
6     for (char &c : data)
7         c = c ^ key;
8     return data;
9 }
10 int main()
11 {
12     string payload;
13     char key;
14     cout << "Enter IP Packet Payload: ";
15     getline(cin, payload);
16     cout << "Enter Encryption Key (single character): ";
17     cin >> key;
18     string encrypted = encryptData(payload, key);
19     cout << "\n--- IPsec ESP Simulation ---\n";
20     cout << "Original Payload : " << payload << endl;
21     cout << "Encrypted Payload: " << encrypted << endl;
22     string decrypted = encryptData(encrypted, key);
23     cout << "Decrypted Payload: " << decrypted << endl;
24     return 0;
25 }
```

Output

```
Enter IP Packet Payload: SecureData
Enter Encryption Key (single character): K

--- IPsec ESP Simulation ---
Original Payload : SecureData
Encrypted Payload: .(>9.*7*
Decrypted Payload: SecureData

=== Code Execution Successful ===
```

3. Design a program to manage Security Associations (SA) in IPsec, including creation, lookup, and deletion operations.

AIM:

To create, search, display and delete IPsec Security Associations

CODING:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
struct SecurityAssociation
```

```
{
```

```
    int spi;
```

```
    string source;
```

```
    string destination; string protocol;
```

```
};
```

```

vector<SecurityAssociation> saTable;

void createSA()
{
    SecurityAssociation sa;

    cout << "Enter SPI: ";

    cin >> sa.spi;

    cout << "Enter Source IP: ";

    cin >> sa.source;

    cout << "Enter Destination IP: ";

    cin >> sa.destination;

    cout << "Enter Protocol (AH/ESP): ";

    cin >> sa.protocol;

    saTable.push_back(sa);

    cout << "Security Association Created Successfully.\n";
}

void lookupSA()
{
    int spi;

    cout << "Enter SPI to lookup: ";

    cin >> spi;

    for (auto sa : saTable)
    {

```

```

    if (sa.spi == spi)
    {
        cout << "\nSA Found\n";

        cout << "SPI      : " << sa.spi << endl;

        cout << "Source IP  : " << sa.source << endl;

        cout << "Destination : " << sa.destination << endl;

        cout << "Protocol   : " << sa.protocol << endl;

        return;
    }
}

cout << "SA Not Found.\n";
}

void deleteSA()
{
    int spi;

    cout << "Enter SPI to delete: ";

    cin >> spi;

    for (auto it = saTable.begin(); it != saTable.end(); ++it)
    {
        if (it->spi == spi)
        {
            saTable.erase(it);

            cout << "Security Association Deleted.\n";

```

```

        return;
    }
}

cout << "SA Not Found.\n";
}

void displaySA()
if (saTable.empty())
{
    cout << "No Security Associations available.\n";

    return;
}

cout << "\n--- Security Association Table ---\n";

for (auto sa : saTable)
{
    cout << "SPI: " << sa.spi

        << " | Source: " << sa.source

        << " | Destination: " << sa.destination

        << " | Protocol: " << sa.protocol << endl;
}
}

int main()
{

```



```

int choice;

do

{
    cout << "\n===== IPSec SA Management =====\n";
    cout << "1. Create SA\n";
    cout << "2. Lookup SA\n";
    cout << "3. Delete SA\n";
    cout << "4. Display SA Table\n";
    cout << "5. Exit\n";
    cout << "Enter choice: ";
    cin >> choice
    switch (choice)
    {
        case 1: createSA(); break;
        case 2: lookupSA(); break;
        case 3: deleteSA(); break;
        case 4: displaySA(); break;
        case 5: cout << "Program Terminated.\n"; break;
        default: cout << "Invalid Choice.\n";
    }
} while (choice != 5);

return 0;

```

}

OUTPUT:

main.cpp	Run	Output
<pre>1 #include <iostream> 2 #include <vector> 3 using namespace std; 4 struct SecurityAssociation 5 { 6 int spi; 7 string source; 8 string destination; 9 string protocol; 10 }; 11 vector<SecurityAssociation> saTable; 12 void createSA() 13 { 14 SecurityAssociation sa; 15 cout << "Enter SPI: "; 16 cin >> sa.spi; 17 cout << "Enter Source IP: "; 18 cin >> sa.source; 19 cout << "Enter Destination IP: "; 20 cin >> sa.destination; 21 cout << "Enter Protocol (AH/ESP): "; 22 cin >> sa.protocol; 23 saTable.push_back(sa); 24 cout << "Security Association Created Successfully\n"; 25 } 26 void lookupSA() 27 { 28 int spi; 29 cout << "Enter SPI to lookup: ";</pre>	Run	Clear <pre>===== IPSec SA Management ===== 1. Create SA 2. Lookup SA 3. Delete SA 4. Display SA Table 5. Exit Enter choice: 1 Enter SPI: 1001 Enter Source IP: 92.168.1.10 Enter Destination IP: 192.168.1.20 Enter Protocol (AH/ESP): ESP Security Association Created Successfully. ===== IPSec SA Management ===== 1. Create SA 2. Lookup SA 3. Delete SA 4. Display SA Table 5. Exit Enter choice:</pre>

```

main.cpp
31     for (auto sa : saTable)
32     {
33         if (sa.spi == spi)
34         {
35             cout << "\nSA Found\n";
36             cout << "SPI          : " << sa.spi << endl;
37             cout << "Source IP    : " << sa.source << endl;
38             cout << "Destination : " << sa.destination << endl;
39             cout << "Protocol    : " << sa.protocol << endl;
40             return;
41         }
42     }
43     cout << "SA Not Found.\n";
44 }
45 void deleteSA()
46 {
47     int spi;
48     cout << "Enter SPI to delete: ";
49     cin >> spi;
50     for (auto it = saTable.begin(); it != saTable.end()
51         ; ++it)
52     {
53         if (it->spi == spi)
54         {
55             saTable.erase(it);
56             cout << "Security Association Deleted.\n";
57         }
58     }
59 }

```

```

main.cpp
63     if (saTable.empty())
64     {
65         cout << "No Security Associations available.\n";
66         return;
67     }
68     cout << "\n--- Security Association Table ---\n";
69     for (auto sa : saTable)
70     {
71         cout << "SPI: " << sa.spi
72             << " | Source: " << sa.source
73             << " | Destination: " << sa.destination
74             << " | Protocol: " << sa.protocol << endl;
75     }
76 }
77 int main()
78 {
79     int choice;
80     do
81     {
82         cout << "\n==== IPsec SA Management ==== \n";
83         cout << "1. Create SA\n";
84         cout << "2. Lookup SA\n";
85         cout << "3. Delete SA\n";
86         cout << "4. Display SA Table\n";
87         cout << "5. Exit\n";
88         cout << "Enter choice: ";
89         cin >> choice;
90         switch (choice)
91         {
92             case 1: createSA(); break;

```

```

main.cpp
73 // | Destination:  sa.destination
74 << " | Protocol: " << sa.protocol << endl;
75 }
76 }
77 int main()
78 {
79     int choice;
80     do
81     {
82         cout << "\n==== IPsec SA Management ==== \n";
83         cout << "1. Create SA \n";
84         cout << "2. Lookup SA \n";
85         cout << "3. Delete SA \n";
86         cout << "4. Display SA Table \n";
87         cout << "5. Exit \n";
88         cout << "Enter choice: ";
89         cin >> choice;
90         switch (choice)
91         {
92             case 1: createSA(); break;
93             case 2: lookupSA(); break;
94             case 3: deleteSA(); break;
95             case 4: displaySA(); break;
96             case 5: cout << "Program Terminated. \n";
97                     break;
98             default: cout << "Invalid Choice. \n";
99         }
100     } while (choice != 5);
101     return 0;
102 }

```

4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

AIM:

To simulate PGP email security using public-key encryption and digital signatures.

CODING:

```
#include <iostream>
```

```
#include <string>
```

```
#include <functional>
```

```
using namespace std;
```

```
long long modPow(long long base, long long exp, long long mod)
```

```
{
```

```

long long result = 1;

while (exp > 0)
{
    if (exp % 2 == 1)
        result = (result * base) % mod;

    base = (base * base) % mod;

    exp /= 2;
}

return result;
}

long long simpleHash(string message)
{
    hash<string> h;

    return h(message) % 1000;
}

int main()
{
    // Simplified RSA parameters

    long long p = 61;

    long long q = 53;

    long long n = p * q;

    long long phi = (p - 1) * (q - 1);

```

```

long long e = 17;

long long d = 2753;

string message;

cout << "Enter Email Message: ";

getline(cin, message);

// Digital Signature

long long hashValue = simpleHash(message);

long long signature = modPow(hashValue, d, n);

cout << "\n--- PGP Simulation ---\n";

cout << "Message Hash   : " << hashValue << endl;

cout << "Digital Signature: " << signature << endl;

// Demonstration of public-key encryption

cout << "\nPublic Key : (" << e << ", " << n << ")" << endl;

cout << "Private Key : (" << d << ", " << n << ")" << endl;

cout << "\nEmail secured using public-key encryption and digital signature.\n";

// Signature verification

long long verifiedHash = modPow(signature, e, n);

if (verifiedHash == hashValue)

    cout << "Digital Signature Verified Successfully.\n";

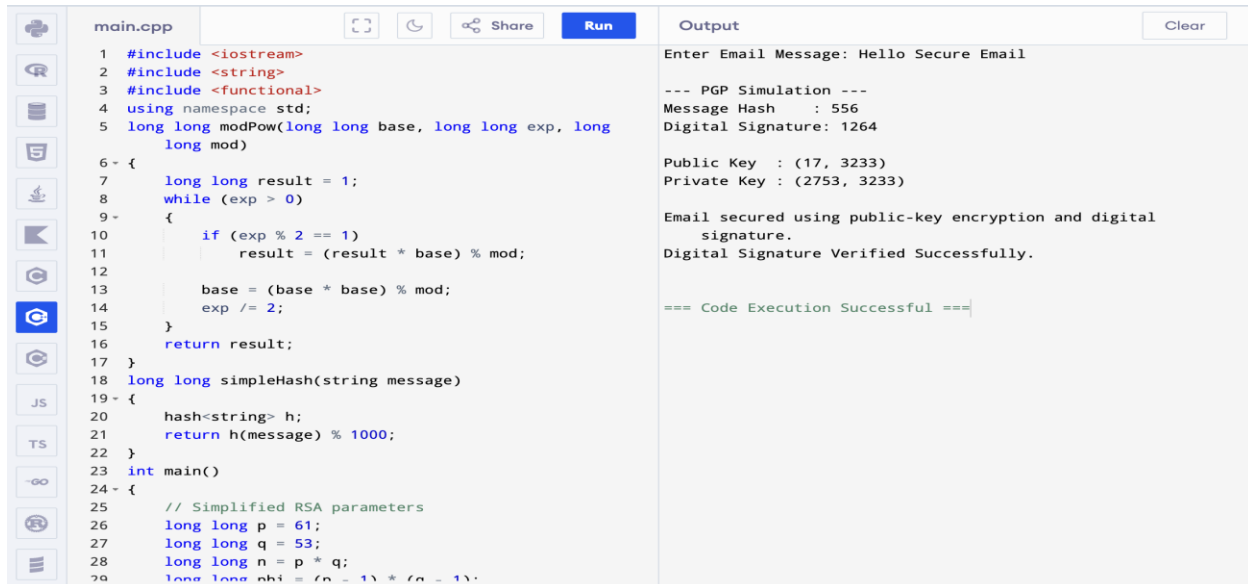
else cout << "Digital Signature Verification Failed.\n";

return 0;

}

```

OUTPUT:



```
main.cpp
1 #include <iostream>
2 #include <string>
3 #include <functional>
4 using namespace std;
5 long long modPow(long long base, long long exp, long long mod)
6 {
7     long long result = 1;
8     while (exp > 0)
9     {
10         if (exp % 2 == 1)
11             result = (result * base) % mod;
12         base = (base * base) % mod;
13         exp /= 2;
14     }
15     return result;
16 }
17 long long simpleHash(string message)
18 {
19     hash<string> h;
20     return h(message) % 1000;
21 }
22 int main()
23 {
24     // Simplified RSA parameters
25     long long p = 61;
26     long long q = 53;
27     long long n = p * q;
28     long long phi = (p - 1) * (q - 1);
29 }
```

Output

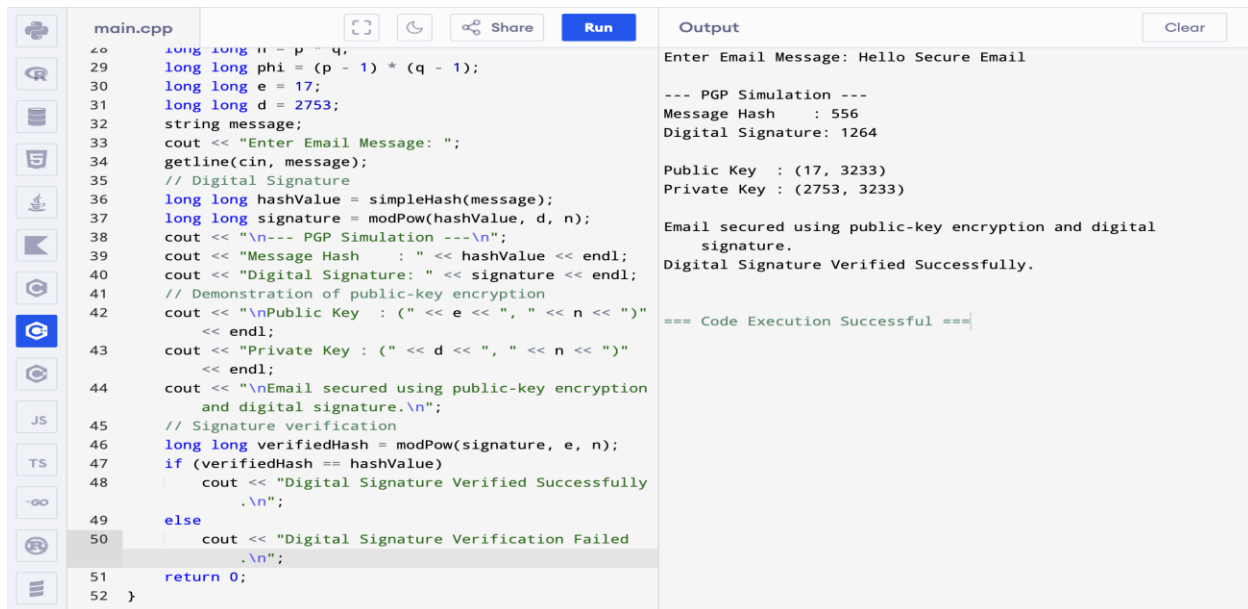
Enter Email Message: Hello Secure Email

--- PGP Simulation ---
Message Hash : 556
Digital Signature: 1264

Public Key : (17, 3233)
Private Key : (2753, 3233)

Email secured using public-key encryption and digital signature.
Digital Signature Verified Successfully.

=== Code Execution Successful ===



```
main.cpp
28 long long n = p * q;
29 long long phi = (p - 1) * (q - 1);
30 long long e = 17;
31 long long d = 2753;
32 string message;
33 cout << "Enter Email Message: ";
34 getline(cin, message);
35 // Digital Signature
36 long long hashValue = simpleHash(message);
37 long long signature = modPow(hashValue, d, n);
38 cout << "\n--- PGP Simulation ---\n";
39 cout << "Message Hash : " << hashValue << endl;
40 cout << "Digital Signature: " << signature << endl;
41 // Demonstration of public-key encryption
42 cout << "\nPublic Key : (" << e << ", " << n << ")"
43 << endl;
44 cout << "Private Key : (" << d << ", " << n << ")"
45 << endl;
46 cout << "\nEmail secured using public-key encryption
47 and digital signature.\n";
48 // Signature verification
49 long long verifiedHash = modPow(signature, e, n);
50 if (verifiedHash == hashValue)
51     cout << "Digital Signature Verified Successfully
52     .\n";
53 else
54     cout << "Digital Signature Verification Failed
55     .\n";
56 return 0;
57 }
```

Output

Enter Email Message: Hello Secure Email

--- PGP Simulation ---
Message Hash : 556
Digital Signature: 1264

Public Key : (17, 3233)
Private Key : (2753, 3233)

Email secured using public-key encryption and digital signature.
Digital Signature Verified Successfully.

=== Code Execution Successful ===

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

AIM:

To classify emails as spam or legitimate using header and content-based filtering.

CODING:

```

#include <iostream>

#include <string>

#include <algorithm>

using namespace std;

int main()
{
    string sender, subject, content;

    int spamScore = 0; cout << "Enter Sender Email: ";

    getline(cin, sender);

    cout << "Enter Subject: ";

    getline(cin, subject);

    cout << "Enter Email Content: ";

    getline(cin, content);

    string text = sender + " " + subject + " " + content;

    transform(text.begin(), text.end(), text.begin(), ::tolower);

    // Header analysis

    if (sender.find("@") == string::npos)

        spamScore += 2;

    // Content analysis

    string keywords[] =

    {

        "win", "winner", "free", "prize",

```



```

        "urgent", "offer", "lottery", "click here", "claim"
    };

    for (string word : keywords)
    {
        if (text.find(word) != string::npos)
            spamScore++;
    }

    cout << "\n--- Email Analysis ---\n";

    cout << "Spam Score: " << spamScore << endl;

    if (spamScore >= 2)
        cout << "Classification: SPAM EMAIL\n";
    else
        cout << "Classification: LEGITIMATE EMAIL\n";

    return 0;
}

```

OUTPUT:

main.cpp

Share

Run

1

#include <iostream>

2

#include <string>

3

#include <algorithm>

4

using namespace std;

5

int main()

6

{

7

string sender, subject, content;

8

int spamScore = 0;

9

cout << "Enter Sender Email: ";

10

getline(cin, sender);

11

cout << "Enter Subject: ";

12

getline(cin, subject);

13

cout << "Enter Email Content: ";

14

getline(cin, content);

15

string text = sender + " " + subject + " " + content;

16

transform(text.begin(), text.end(), text.begin(),

::tolower);

17

// Header analysis

18

if (sender.find("@") == string::npos)

19

spamScore += 2;

20

// Content analysis

21

string keywords[] =

22

{

23

"win", "winner", "free", "prize",

24

"urgent", "offer", "lottery",

25

"click here", "claim"

26

};

27

for (string word : keywords)

28

{

29

if (text.find(word) != string::npos)

30

{

31

spamScore++;

32

}

33

}

34

cout << "\n--- Email Analysis ---\n";

35

cout << "Spam Score: " << spamScore << endl;

36

if (spamScore >= 2)

37

cout << "Classification: SPAM EMAIL\n";

38

else

39

cout << "Classification: LEGITIMATE EMAIL\n";

40

return 0;

41

}

Output

Clear

Enter Sender Email: unknown@gmail.com
Enter Subject: You won a free prize
Enter Email Content: Urgent! Click here to claim your prize.

--- Email Analysis ---
Spam Score: 5
Classification: SPAM EMAIL

=== Code Execution Successful ===

main.cpp

Share

Run

11

cout << "Enter Subject: ";

12

getline(cin, subject);

13

cout << "Enter Email Content: ";

14

getline(cin, content);

15

string text = sender + " " + subject + " " + content;

16

transform(text.begin(), text.end(), text.begin(),

::tolower);

17

// Header analysis

18

if (sender.find("@") == string::npos)

19

spamScore += 2;

20

// Content analysis

21

string keywords[] =

22

{

23

"win", "winner", "free", "prize",

24

"urgent", "offer", "lottery",

25

"click here", "claim"

26

};

27

for (string word : keywords)

28

{

29

if (text.find(word) != string::npos)

30

{

31

spamScore++;

32

}

33

}

34

cout << "\n--- Email Analysis ---\n";

35

cout << "Spam Score: " << spamScore << endl;

36

if (spamScore >= 2)

37

cout << "Classification: SPAM EMAIL\n";

38

else

39

cout << "Classification: LEGITIMATE EMAIL\n";

40

return 0;

41

}

Output

Clear

Enter Sender Email: unknown@gmail.com
Enter Subject: You won a free prize
Enter Email Content: Urgent! Click here to claim your prize.

--- Email Analysis ---
Spam Score: 5
Classification: SPAM EMAIL

=== Code Execution Successful ===